# Analysis of Rendering Algorithms Using Asymptotic Notation

## Aim

To analyze three rendering algorithms having time complexities **O(n)**, **O(n log n)**, and **O(n²)** using asymptotic notation and determine the most efficient algorithm for high-resolution rendering.

## Objective

- Study asymptotic notation.
- Compare the growth rates of three algorithms.
- Analyze their performance for large input sizes.
- Identify the most efficient algorithm.

## Theory

Asymptotic notation describes how an algorithm performs as the input size increases.

**1. O(n)**

- Linear complexity.
- Execution time increases proportionally with input size.
- Best among the three for large datasets.

**2. O(n log n)**

- Log-linear complexity.
- Slightly slower than O(n).
- Commonly used in efficient sorting algorithms like Merge Sort and Heap Sort.

**3. O(n²)**

- Quadratic complexity.
- Execution time increases rapidly as input grows.
- Suitable only for small datasets.

## Algorithm

1. Read the number of input sizes from the user.
2. Repeat for each input size:

- Read the value of **n**.
- Compute:
  - Linear Complexity = **n**
  - Log-Linear Complexity = **n × $\log_2(n)$**
  - Quadratic Complexity = **$n^2$**

3. Display the calculated values in tabular form.
4. Compare the values.
5. Display the conclusion.

# SOURCE CODE SCREENSHOTS:

main.c

```c
#include <stdio.h>
#include <math.h>
int main()
{
    int inputs[] = {10, 100, 1000, 10000};
    int i;
    printf("-----------------------------------------------------------------\n");
    printf(" Input\t\tO(n)\t\tO(n log n)\t\tO(n^2)\n");
    printf("-----------------------------------------------------------------\n");
    for(i = 0; i < 4; i++)
    {
        int n = inputs[i];
        double linear = n;
        double nlogn = n * (log(n) / log(2));
        double quadratic = (double)n * n;
        printf("%5d\t\t%.0f\t\t%.2f\t\t%.0f\n",
               n, linear, nlogn, quadratic);
    }
    printf("\n-----------------------------------------------------------------\n");
    printf("\nObservation:\n");
    for(i = 0; i < 4; i++)
    {
        int n = inputs[i];
        double linear = n;
        double nlogn = n * (log(n) / log(2));
```

Output

```
-----------------------------------------------------------------
 Input      O(n)        O(n log n)      O(n^2)
-----------------------------------------------------------------
   10       10      33.22       100
  100       100     664.39      10000
 1000       1000        9965.78     1000000
10000       10000       132877.12       100000000

-----------------------------------------------------------------

Observation:
For n = 10 : O(n) < O(n log n) < O(n^2)
For n = 100 : O(n) < O(n log n) < O(n^2)
For n = 1000 : O(n) < O(n log n) < O(n^2)
For n = 10000 : O(n) < O(n log n) < O(n^2)

Conclusion:
As the input size increases, O(n) grows the slowest and is the most efficient.
O(n log n) is moderately efficient, while O(n^2) grows rapidly.


=== Code Execution Successful ===
```

main.c

```c
    double linear = n;
    double nlogn = n * (log(n) / log(2));
    double quadratic = (double)n * n;
    printf("%5d\t\t%.0f\t\t%.2f\t\t%.0f\n",
           n, linear, nlogn, quadratic);
}
printf("\n-----------------------------------------------------------------\n");
printf("\nObservation:\n");
for(i = 0; i < 4; i++)
{
    int n = inputs[i];
    double linear = n;
    double nlogn = n * (log(n) / log(2));
    double quadratic = (double)n * n;
    if(linear < nlogn && nlogn < quadratic)
    {
        printf("For n = %d : O(n) < O(n log n) < O(n^2)\n", n);
    }
}
printf("\nConclusion:\n");
printf("As the input size increases, O(n) grows the slowest and is the most efficient
    .\n");
printf("O(n log n) is moderately efficient, while O(n^2) grows rapidly.\n");
return 0;
}
```

Output

```
-----------------------------------------------------------------
 Input      O(n)        O(n log n)      O(n^2)
-----------------------------------------------------------------
   10       10      33.22       100
  100       100     664.39      10000
 1000       1000        9965.78     1000000
10000       10000       132877.12       100000000

-----------------------------------------------------------------

Observation:
For n = 10 : O(n) < O(n log n) < O(n^2)
For n = 100 : O(n) < O(n log n) < O(n^2)
For n = 1000 : O(n) < O(n log n) < O(n^2)
For n = 10000 : O(n) < O(n log n) < O(n^2)

Conclusion:
As the input size increases, O(n) grows the slowest and is the most efficient.
O(n log n) is moderately efficient, while O(n^2) grows rapidly.


=== Code Execution Successful ===
```

**OUTPUT:**

```
------------------------------------------------------------
 Input          O(n)          O(n log n)              O(n^2)
------------------------------------------------------------
   10           10            33.22          100
  100           100           664.39         10000
 1000           1000          9965.78                1000000
10000           10000         132877.12              100000000
------------------------------------------------------------
```

Observation:

For n = 10 : O(n) < O(n log n) < O(n^2)

For n = 100 : O(n) < O(n log n) < O(n^2)

For n = 1000 : O(n) < O(n log n) < O(n^2)

For n = 10000 : O(n) < O(n log n) < O(n^2)

Conclusion:

As the input size increases, O(n) grows the slowest and is the most efficient.

O(n log n) is moderately efficient, while O(n^2) grows rapidly.

## Analysis :

The program computes the theoretical values of three different asymptotic complexities for multiple input sizes.

| **Input (n)** | **O(n)** | **O(n log n)** | **O(n²)** |
|---|---|---|---|
| 10 | 10 | 33.22 | 100 |
| 100 | 100 | 664.39 | 10,000 |
| 1000 | 1000 | 9965.78 | 1,000,000 |
| 10000 | 10000 | 132877.12 | 100,000,000 |

## Observation

- **O(n)** increases linearly and remains the smallest for all input sizes.
- **O(n log n)** grows slightly faster than O(n) but is still efficient for large inputs.
- **O(n²)** increases very rapidly, making it unsuitable for high-resolution or large datasets.

As the input size increases from **10** to **10000**, the difference between the three complexities becomes significantly larger. This demonstrates how growth rate affects algorithm performance.

## Performance Comparison:

| Complexity | Growth Rate | Performance |
|---|---|---|
| **O(n)** | Linear | Excellent |
| **O(n log n)** | Log-Linear | Very Good |
| **O(n²)** | Quadratic | Poor for Large Inputs |

## Time Complexity:

| Algorithm | Time Complexity |
|---|---|
| Linear | **O(n)** |
| Log-Linear | **O(n log n)** |
| Quadratic | **O(n²)** |

## Space Complexity:

**Space Complexity = O(1)**

**Growth Rate Impact**

- For small input sizes, the difference between the algorithms is relatively small.
- As the input size increases, the quadratic algorithm (**O(n²)**) grows much faster than the other two.
- The linear algorithm (**O(n)**) scales efficiently even for very large inputs.

- Therefore, applications such as gaming engines that process high-resolution graphics should prefer algorithms with lower growth rates to achieve better performance.

## Documentation:

### Software Requirements

- Turbo C / Dev C++ / Code::Blocks / VS Code
- GCC Compiler
- C Programming Language

### Header Files Used

- stdio.h – Input and Output operations.
- math.h – Used for logarithmic calculations (log()).

### Functions Used

- scanf() – Reads user input.
- printf() – Displays the output.
- log() – Computes the logarithm used in calculating **O(n log n)**.